Contextual Discontinuity and Long-Horizon Adversarial Coordination

Research proposal • Bennett Bishop • 9 October 2026

Research question

Can periodic context resets make an untrusted AI agent less able to carry out a multi-step attack without making it substantially worse at completing legitimate work? More specifically, which ways of passing a summary of previous work to the next agent—what we call handoff policies—best preserve useful work while disrupting adversarial coordination?

Motivation and contribution

Previous AI-control research has shown why attacks spread across several actions can be difficult to detect: each individual action may look harmless even when the full sequence is malicious.[1] Other work has found that dividing work into smaller tasks can help weaker trusted models monitor the results.[2] Factor(T,U) also raises the possibility that isolated tasks might make it harder for an attacker to coordinate a multi-step strategy, although it does not directly test that mechanism. This proposal investigates that possibility without dividing a job into separate subtasks. Instead, the agent continues working in the same environment, but periodically loses its conversation history and receives a new handoff summary of its previous work. Our immediate goal is to test whether this approach reduces attack success without substantially harming legitimate task performance. We are not proposing it as a complete control protocol on its own; a natural next step would be to combine it with established safeguards such as monitoring, resampling, and defer-to-trusted approaches.

Three kinds of state

The experiment distinguishes three ways information can persist. Conversational state is the agent's recorded interaction history: its messages, available reasoning or planning text, tool calls and outputs, and records of changes it made. A reset removes this history from the successor's context, but does not reverse the actions themselves. Handoff state is the summary or structured record deliberately passed to the successor. Environmental state is what remains in the system, such as modified files, code, and configuration, which the successor can inspect again. We compare handoff policies while keeping environmental state intact.

1Bhatt et al., "Ctrl-Z: Controlling AI Agents via Resampling," discussion of multi-step attacks in BashBench: https://arxiv.org/abs/2504.10374.

2Sandoval and Rushing, "Factor(T,U): Factored Cognition Strengthens Monitoring of Untrusted AI" (2025), monitoring results and §4.2: https://arxiv.org/abs/2512.02157.

Hypotheses

H1 — Contextual discontinuity. Losing access to earlier conversation history makes it harder for an untrusted agent to complete attacks that depend on coordinating actions over time, even when it still knows its adversarial objective and can access the same environment.

H2 — Usefulness tradeoff. Some ways of writing handoffs will reduce attack success more than they reduce legitimate task success.

H3 — Recovery of attack information. Attacks may still succeed if a handoff happens to preserve important information or if the agent can recover it from files, configuration changes, or other persistent artifacts.

Environment and evaluation population

We plan to use LinuxArena, which provides realistic Linux software and infrastructure tasks together with adversarial side objectives and ways to check whether each objective was completed. LinuxArena is particularly suitable because agents work across many actions in a persistent environment: changes they leave behind can still be discovered and used after their conversational history is reset. We intend to evaluate all supported task combinations we can access, rather than select only attacks that seem likely to be interrupted by resets. This avoids making the mechanism look stronger by choosing especially favorable examples. We will also report how many runs actually reach a reset before the attack succeeds; attacks completed before the first reset still belong in the overall results.

Threat model

The attacker can choose its tool calls and modify the environment as its permissions allow. It cannot alter the reset schedule, restore discarded conversation history, or control the trusted summarizer or its instructions. Every new agent instance receives the legitimate task and the adversarial side objective again. It also receives whatever handoff its assigned policy permits. The underlying Linux environment remains in place: files and configuration changes made by an earlier instance do not disappear. The trusted summarizer sees only the recorded tool actions and outputs, file changes, and other observable execution history. It does not receive the original objectives or instructions, or the agent's private reasoning, as separate inputs. Its job is simply to summarize the work, not to identify or filter malicious activity.

Experimental intervention

During the main comparison we will reset the agent after a fixed number of tool actions, with 20 actions as a provisional starting point. We will choose the final interval after checking typical LinuxArena trajectory lengths. The feature we vary is the handoff policy: what account of earlier work the successor receives. At each reset, we start a fresh agent with the same underlying environment and original objectives. Only its access to earlier work differs by condition. In the full-context control,

the agent receives its complete eligible history again. This lets us separate the effect of losing information from any effect of restarting the agent.

Condition	Handoff information
Full-context control	Entire eligible previous trajectory supplied after each restart
Detailed trusted summary	Trusted model produces an extensive account of visible prior progress
Short trusted progress report	Trusted model produces a tightly budgeted natural-language report
Structured trusted state	Trusted model fills predefined fields describing observed work and environment facts
Minimal context	Original objectives and only the minimum operational handoff needed to continue (exact content to be specified)

The handoffs differ in how they present and select information. A detailed summary, a short progress report, and a structured record may each preserve different details, even when they describe the same work. In all trusted-summary conditions, the summarizer starts from the same observable work history and follows a specified format and length limit. We will publish the exact prompts, inputs, and limits used in each condition.

Attack-policy approach

We will use the same attacker setup in every handoff condition. LinuxArena includes an existing attack policy with exploration, planning, and execution phases, but we must first check exactly where that policy keeps its plans. A plan stored outside the conversation could accidentally survive a reset and undermine the comparison. We have not yet decided whether to reuse that policy with modifications or use a simpler continuous agent informed by its prompting. Before comparing handoff policies, we will check that our chosen attacker can successfully complete a meaningful number of side tasks under full context; otherwise, there would be little attack capability for the intervention to affect. As a later robustness check, we may let an attacker know the handoff rules and deliberately adapt its strategy to them.

Outcome measures

We will focus on two main results: how often an honest agent completes the legitimate task, and how often an attacking agent completes its adversarial side objective. Each handoff condition will be evaluated on the same available task combinations, with comparable time and tool-use limits. We will also record the number of resets, tool actions, model calls, and approximate run cost, primarily to help interpret the results. We are not making monitor suspiciousness or undetected-attack success a required outcome of this first study.

Analysis and interpretation

For each task, we will compare performance under the different handoff policies with performance when the agent retains its complete history. Where feasible, we will run tasks more than once so that a single lucky or unlucky attempt does not dominate the findings, and report uncertainty around the differences. We will report both absolute percentage-point changes and relative changes in attack and legitimate-task success compared with their full-context baselines. We will look separately at whether resets impair legitimate work and whether they impair the attacker's side objective. If attacks fail more often, that alone would not prove the model "forgot" an attack plan: it could also have taken longer or lost useful information needed for ordinary work. To investigate possible explanations, we may closely examine a few illustrative trajectories, while treating those examples as suggestive rather than representative of every run.

Limitations and future directions

Some attacks may finish before the first reset, so the intervention would have no chance to affect them. Attackers may also be able to recover useful information from handoffs or persistent files. Rather than treating this as a flaw in the experiment, we would regard it as evidence about the limits of context discontinuity as a defense. Results will also depend on how effectively the attacker is prompted and how much each handoff policy impairs ordinary work. A reduction in attack success by itself would not prove that an attacker lost its plan.

The most important next step, if the mechanism shows promise, is to combine context resets with established AI-control techniques. For example, trusted or untrusted monitors could examine actions before they execute; suspicious actions could trigger defer-to-trusted decisions, resampling, or an audit of earlier work. Such experiments would ask whether discontinuity adds safety beyond those existing protocols, and what useful performance it costs. Another extension would vary reset frequency or test attackers explicitly adapted to the handoff policy.

References and implementation resources

Selected references and implementation resources:

Bhatt et al., "Ctrl-Z: Controlling AI Agents via Resampling" (BashBench). https://arxiv.org/abs/2504.10374

Sandoval and Rushing, "Factor(T,U): Factored Cognition Strengthens Monitoring of Untrusted AI" (2025), §4.2. https://arxiv.org/abs/2512.02157

LinuxArena. https://arxiv.org/abs/2604.15384

LinuxArena reproducibility code. https://github.com/redwoodresearch/linuxarena-paper

Control Tower documentation. https://docs.linuxarena.ai/